

Two missed Abs contracts

in AsymptoticAnalysis

Incorrect real outputs and lost differentiability

Repository VladimirReshetnikov/Asymptotic

Pinned revision 651f2029d0b2cd4da9e4dfdf1f4275a124d23b99

Review date 10 September 2026

Deliverables Article, conservative patch generator, exact reference
implementation, independent checks, prospective Wolfram tests.

Principal result. Two source-level defects survive the prior-review exclusion filter. The absolute-value shortcut can turn complex intermediate jets into an incorrect real, apparently exact answer. Separately, a valid modulus magnitude estimate is allowed to inherit a derivative contract that it does not preserve. The latter has an exact Fourier-source witness with infinitely many cusps approaching the expansion point.

Evidence boundary. The mathematical counterexamples and the reference algorithm were checked independently in Python/SymPy. The Wolfram kernel service returned upstream errors; no native package execution, native patch validation, or full-suite pass is claimed.

Contents

1	Executive assessment	2
2	Snapshot, scope, and exclusion method	2
2.1	Source identity	2
2.2	What was inspected	2
2.3	Prior-review suppression	3
3	The contracts that the implementation must distinguish	4
3.1	Power-log value information	4
3.2	Magnitude versus differentiability	4
4	ABS-01: a real leading sign is not a modulus identity	4
4.1	The source rule	4
4.2	A polynomial-unit counterexample	5
4.3	A logarithmic example that exposes the scale boundary	5
4.4	Prospective public characterizations	6
5	ABS-02: a valid modulus bound can destroy a derivative contract	6
5.1	The source path	6
5.2	An exact source inside the Fourier model	7
5.3	Infinitely many simple zeros and cusps	7
5.4	Prospective public trace	8
6	Repairs: conservative first, richer only with proved contracts	9
6.1	Patch A: guard the signed-real shortcut	9
6.2	Patch B: lower the derivative contract at the observable boundary	9
6.3	A richer value algorithm: square the norm, then take the positive root	9
6.4	Recovering regularity without overclaiming it	10
7	Artifacts, test results, and implementation costs	10
7.1	What is supplied	10
7.2	Independent checks actually run	11
7.3	Scope and complexity of the reference implementation	11
8	Focused implementation and acceptance plan	12
8.1	First milestone: remove false successes	12
8.2	Second milestone: restore justified capabilities	12
8.3	Third milestone: make the new behavior understandable	12
9	Limitations and conclusion	12
A	Reproduction and interpretation	13
A.1	Independent checks	13
A.2	Native characterization, not a recorded result	13
A.3	Stage the source edits	14
B	Source locator and evidence register	14

1 Executive assessment

The two findings concern different meanings of correctness. **ABS-01 is a value error**: a complex argument with a positive real leading coefficient is treated as though taking its modulus were the identity. Two such incorrect intermediate results can cancel their imaginary terms, so the final real-coefficient validator cannot detect the error. **ABS-02 is a regularity error**: a correct estimate of the form $|r| = \mathcal{O}(R)$ is allowed to retain a classical derivative contract, even when $|r|$ has infinitely many nondifferentiable points.

Both deductions use the current shared `fwdAbs` primitive and the public observable/differentiation paths, rather than an invented replacement implementation. Source locations and proposed native characterizations are provided below. Nevertheless, the distinction between a source trace and an observed kernel result remains important: all public Wolfram outputs in this article are *predictions to be tested*, not measurements. The independent mathematical identities are established separately.

ID	Priority	Failure and immediate response	Evidence
ABS-01	High correctness	Reject the signed-real shortcut when retained coefficients are not proved real; later add a genuine modulus-jet adapter.	Source trace; exact counterexamples
ABS-02	High contract integrity	Do not copy differentiability through an uncertain zero of <code>Abs</code> ; conservatively lower the output derivative contract.	Source trace; exact smooth-source cusp proof

The supplied patch is deliberately narrower than a full complex-modulus implementation. It modifies two modular files, refuses the unsafe shortcut, and conservatively lowers the derivative contract for nonexact variable-dependent `Abs` observables. Its transformation logic passed exact-anchor fixture checks, but the modified package has not been run in a Wolfram kernel. The richer mathematical repair is illustrated by a tested, log-free Python reference implementation based on the squared modulus.

No additional performance defect or measured speedup is claimed. The performance discussion concerns the cost and admissible scope of these *new repairs*, not an attempt to relabel already recorded performance recommendations as fresh findings.

2 Snapshot, scope, and exclusion method

2.1 Source identity

The reviewed snapshot is `651f2029d0b2cd4da9e4dfdf1f4275a124d23b99`, whose commit time is 10 September 2026 at 13:42:02 UTC. Its commit message is “Record affine ranges and truncation bound transport in the article.” All source links in this report are commit-pinned; none depends on the future contents of the mutable `main` branch. The snapshot was retrieved through the GitHub connector. A complete working checkout was not available in the execution container, so no claim is made that a repository-wide local search or build was performed.[1]

2.2 What was inspected

The investigation ranged beyond the two files ultimately patched. Selected source ranges were examined in the sparse power-log core and series operations; Gamma and Barnes forward/inverse

formulas and residual checks; exact source-coordinate reconstruction; reciprocal-log calculus; Dirichlet special-function bounds; exact special-function identities and parameterized adapters; native special-function tree transport; Fourier coefficient arithmetic; interval inverse certificates; several Mathics adapters; and the portable acceptance verifier. The analysis of some files was complete and of others was range-based. This is a targeted source review, not a statement-coverage audit.

The eventual findings concentrate on a small shared boundary because its failures are both mathematically sharp and distinct from the accumulated backlog. Inspection without a new finding is not a correctness proof for a module. In particular, no broad claim about all inverse families, all native **Series** trees, all Mathics behavior, or all numerical certificates follows from this work.

2.3 Prior-review suppression

The repository’s review index describes 36 review packages in four waves and 231 report entries before deduplication. The consolidated status register and the wave-3/wave-4 intake documents were used as the primary exclusion baseline. Additional original finding ledgers were checked for reviews 1, 4, 5, 6, 7, and 8, together with the original review-25 summary. This does *not* mean every paragraph of all 36 original articles was read.[5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

This distinction mattered in practice. A native trigonometric-transport candidate initially appeared promising, but an original ledger identified it as review 4’s R01. It is therefore absent from the findings, patch, and recommendations in this article. Suppression also applies to proposals, not merely to defects already fixed upstream.

The nearest existing obligations to ABS-01 are C07 and review 4’s R02: they concern coefficient-*t*/realness boundaries. ABS-01 is not another example of a complex final coefficient escaping validation. Its final coefficient array is real and therefore passes that validation; the value was corrupted *inside a nonholomorphic operation before collection*. The nearest broad obligations to ABS-02 are C13/C16. The new witness does not rely on an opaque custom **Series** handler, a false user declaration, a discontinuous input function, or a malformed raw object. It uses an explicit smooth Fourier source, valid derivative bounds, and a concrete derivative-contract transfer through an uncertain modulus zero.

Candidate	Nearest baseline references	Exclusion decision
ABS-01	C07; R4 R02; W3-06	Retained: new incorrect-real-output mechanism at <code>fwdAbs</code> .
ABS-02	C13; C16; X04	Retained: new explicit classical-regularity counterexample after a legitimate derivative declaration.
Other native transport candidate	R4 R01	Suppressed; no new finding or patch is distributed for it.

The novelty conclusion is relative to the retrieved registers and the specified original ledgers. It is not a guarantee that an unindexed sentence elsewhere in the repository has never suggested a related idea.

3 The contracts that the implementation must distinguish

3.1 Power-log value information

For a positive local variable $u \rightarrow 0^+$, the core represents a jet by

$$J = (T, P, D), \quad A(u) = \sum_{(\alpha, Q) \in T} u^\alpha Q(\ell), \quad \ell = \log u,$$

with a magnitude remainder

$$f(u) = A(u) + \mathcal{O}\left(u^P(1 + |\log u|)^D\right).$$

The rows are exponent/coefficient-polynomial pairs. A tail power $P = \infty$ represents no omitted remainder in this finite jet model. The public `PowerLogRemainder` descriptor explicitly denotes a magnitude class, not the actual signed residual.[2]

Intermediate arithmetic deliberately permits complex coefficients: a real final expression can be a sum of complex intermediate expressions. The current `realCoefficientRows` routine checks the *collected* coefficients at representation boundaries. This policy is useful and should not be replaced by an indiscriminate ban on complex intermediate arithmetic.[2]

The policy is valid only when each preceding transformation is mathematically valid on the intermediate values. Addition and multiplication do not become invalid merely because their operands are complex. In contrast, the identity $|z| = z$ is not a complex identity, even near the positive real axis.

3.2 Magnitude versus differentiability

A derivative contract adds information that a magnitude remainder does not contain. In the simplest local chart, a contract through order N means that the relevant classical derivatives exist on a sufficiently small punctured interval and obey appropriate estimates, for example

$$r^{(k)}(u) = \mathcal{O}(u^{P-k}), \quad 0 \leq k \leq N.$$

The package's `SeriesDifferentiate` checks an available or explicitly declared `RemainderDerivativeOrder` before differentiating. It then transports that order through the derivative calculation. This is materially stronger than an almost-everywhere derivative bound.[3]

The modulus is globally Lipschitz:

$$||z| - |w|| \leq |z - w|, \quad z, w \in \mathbb{C}. \quad (1)$$

Thus $r = \mathcal{O}(R)$ always implies $|r| = \mathcal{O}(R)$. But (1) says nothing about differentiability at zeros of r . A smooth real function crossing zero simply acquires a cusp under `Abs`. A derivative-capability field must therefore describe the regularity of the *new remainder*, not merely repeat the capability of the input remainder.

4 ABS-01: a real leading sign is not a modulus identity

4.1 The source rule

The relevant primitive is in `src/Kernel/AsymptoticAnalysis.wl`, lines 460–465. Its operative logic is:

```

fwdAbs[j : {T_, P_, D_}, ell_, ass_] := Module[{q, c, degree},
  If[T === {}, Return[j, Module]];
  q = T[[1, 2]]; degree = polyDegree[q, ell];
  c = (-1)^degree Coefficient[q, ell, degree];
  Which[provablyPositive[c, ass], j,
    provablyNegative[c, ass], pScale[j, -1, ell, ass],
    True, fail["UnprovedSign", ...]];

```

The factor $(-1)^{\deg Q}$ correctly accounts for $\ell \rightarrow -\infty$ when a *real* leading polynomial has an eventual sign. The defect is not that sign calculation. It is that no condition here establishes realness of the remaining coefficients, or even of all coefficients of the leading polynomial.

Both ordinary forward expansion and `seriesJetApply` delegate `Abs` to this helper. The latter has a dedicated `Abs` arm before its generic unary-observable admission logic. After applying the observable, `SeriesObservable` constructs a result with `seriesMake`, where realness is checked only after all outer additions have been collected.[2, 3]

4.2 A polynomial-unit counterexample

Let $u > 0$. Consider

$$F(u) = |1 + iu| + |1 - iu| = 2\sqrt{1 + u^2}. \quad (2)$$

The two exact input jets at the modulus nodes are

$$J_+ = (\{(0, 1), (1, i)\}, \infty, 0), \quad J_- = (\{(0, 1), (1, -i)\}, \infty, 0).$$

For each, the helper sees a positive leading coefficient 1 and returns the jet unchanged. The surrounding addition consequently yields the exact constant jet 2. Its only coefficient is real; the final real-coefficient check has no evidence left with which to reject it.

The actual series is

$$F(u) = 2 + u^2 - \frac{u^4}{4} + \frac{u^6}{8} + \mathcal{O}(u^8). \quad (3)$$

In particular,

$$\lim_{u \rightarrow 0^+} \frac{F(u) - 2}{u^2} = 1.$$

A result 2 with zero remainder is false. Even a result $2 + \mathcal{O}(u^4)$ would be false. This example therefore distinguishes genuine value correctness from the more limited property “all final coefficients are real.”

The phenomenon is not isolated. For any $a > 0$, $b \in \mathbb{R} \setminus \{0\}$, and positive integer n ,

$$|a + ibu^n| + |a - ibu^n| = 2a + \frac{b^2}{a}u^{2n} + \mathcal{O}(u^{4n}).$$

The signed shortcut instead produces $2a$. The independent checks cover 27 exact rational parameter/exponent choices from this family.

4.3 A logarithmic example that exposes the scale boundary

A second witness makes the missing information even more conspicuous:

$$H(u) = |\log u + i| + |\log u - i| = 2\sqrt{(\log u)^2 + 1}. \quad (4)$$

For $0 < u < 1$, write $t = -\log u > 0$. Each leading polynomial $\ell \pm i$ has degree one and leading coefficient 1. The helper's eventual-sign calculation produces -1 , so it negates both jets. Their imaginary constants cancel, predicting the exact result $-2\log u = 2t$.

But

$$H(u) = 2t + \frac{1}{t} - \frac{1}{4t^3} + \mathcal{O}(t^{-5}), \quad \lim_{t \rightarrow \infty} t(2\sqrt{t^2 + 1} - 2t) = 1.$$

The first omitted contribution is a reciprocal logarithm. It is larger than u^q for every fixed $q > 0$, because $e^{qt}/t \rightarrow \infty$. A polynomial-log-only answer cannot silently discard it while claiming an exact result or a positive-power tail.

This witness also clarifies the correct behavior of a conservative fix: refusal is legitimate when the coefficient algebra cannot represent the modulus. Returning a convenient but false power-log expression is not. A richer reciprocal-log representation is an implementation extension, not a justification for the current shortcut.

4.4 Prospective public characterizations

The distributed native script uses explicit package selection so that a native fallback cannot conceal the path being tested:

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  u, {u, 0, 4}, "Backend" -> "Package"];

AsymptoticAnalysis'SeriesObservable[
  s, Abs[1 + I z] + Abs[1 - I z], z]

AsymptoticAnalysis'SeriesObservable[
  s, Abs[Log[z] + I] + Abs[Log[z] - I], z]
```

The source predicts incorrect exact expressions 2 and $-2\log u$, respectively, provided these public calls reach the inspected helper without an earlier evaluation transformation. *These calls were not executed in a native kernel during the review.* The helper-level counterexample does not depend on that last integration detail: the displayed arrays themselves satisfy its input representation and deterministically take the unsafe branches.

The proposed regression set includes both public calls and direct helper controls. It also verifies that cancellation *inside one argument* remains valid, as in

$$|(1 + iu) + (1 - iu)| = 2.$$

The new guard belongs at the modulus boundary, after its own argument has been collected, not at every complex arithmetic intermediate.

5 ABS-02: a valid modulus bound can destroy a derivative contract

5.1 The source path

There are three relevant steps. First, `fwdAbs` returns an empty-row finite-tail jet unchanged. This is sound as a magnitude estimate: $r = \mathcal{O}(R)$ implies $|r| = \mathcal{O}(R)$. Second, `SeriesObservable` replaces the jet in the input representation but otherwise retains the representation fields. That includes `RemainderDerivativeOrder`. Third, `seriesDerivative` trusts that retained field when deciding whether the next derivative is allowed.[2, 3]

The defect is specifically the second step. Fixing ABS-01 by checking that all retained coefficients are real does not fix ABS-02: an empty coefficient list vacuously passes such a check, and the counterexample below is entirely real.

5.2 An exact source inside the Fourier model

Use the exact source function

$$f(x) = x + x^2 \sin(\log x), \quad x > 0. \quad (5)$$

Its derivatives are

$$f'(x) = 1 + x(2 \sin(\log x) + \cos(\log x)), \quad (6)$$

$$f''(x) = \sin(\log x) + 3 \cos(\log x). \quad (7)$$

For $0 < x < 1/4$, the elementary bounds $|f'(x) - 1| \leq 3x < 3/4$ and $|f''(x)| \leq 4$ show that f is strictly increasing and has a smooth inverse g on a target interval adjoining zero. Also $f(x)/x \rightarrow 1$, so $g(y) \sim y$.

Proposition 1. *The remainder $R(y) = g(y) - y$ satisfies a valid classical derivative contract through order two:*

$$R(y) = \mathcal{O}(y^2), \quad R'(y) = \mathcal{O}(y), \quad R''(y) = \mathcal{O}(1).$$

Proof. Set $x = g(y)$. The exact equation gives $g(y) - y = -x^2 \sin(\log x) = \mathcal{O}(x^2) = \mathcal{O}(y^2)$. Inverse differentiation gives

$$g'(y) = \frac{1}{f'(x)}, \quad g''(y) = -\frac{f''(x)}{f'(x)^3}.$$

The lower bound $f'(x) > 1/4$ and (6) imply $g'(y) - 1 = \mathcal{O}(x) = \mathcal{O}(y)$. Equation (7) then gives $g''(y) = \mathcal{O}(1)$. All these are derivatives of the actual smooth inverse, not formal derivatives of an unknown magnitude symbol. $\square$

The source (5) is a monomial leading core with one higher-power finite Fourier coefficient in $\log x$, precisely the form admitted by `AsymptoticFourierInverse`. At cutoff 2, its retained inverse expression is y , with a tail of order y^2 . The Fourier constructor records derivative order zero by default, so the test explicitly supplies the justified order-two declaration from Proposition 1. The general series-data adapter can recover a power-log representation from the finite expression y ; no raw association is fabricated.[4, 3]

5.3 Infinitely many simple zeros and cusps

Let

$$\theta_n = -\arctan(1/2) - n\pi, \quad x_n = e^{\theta_n}, \quad y_n = f(x_n), \quad n \geq 1.$$

These are positive points tending to zero. Since $\tan \theta_n = -1/2$,

$$2 \sin \theta_n + \cos \theta_n = 0,$$

and hence $f'(x_n) = 1$ and $g'(y_n) = 1$. At the same points,

$$f''(x_n) = \sin \theta_n + 3 \cos \theta_n = -5 \sin \theta_n \neq 0,$$

with $|f''(x_n)| = \sqrt{5}$. Consequently

$$g''(y_n) = -f''(x_n), \quad |g''(y_n)| = \sqrt{5}. \quad (8)$$

Thus each zero of $g' - 1$ at y_n is simple.

Proposition 2. *The function $A(y) = |g'(y) - 1|$ is $\mathcal{O}(y)$, but it is not classically differentiable on any complete sufficiently small punctured target interval.*

Proof. The magnitude estimate follows from Proposition 1. Near any y_n , Taylor’s theorem gives

$$g'(y) - 1 = g''(y_n)(y - y_n) + o(|y - y_n|).$$

Therefore A has left and right derivatives $-\sqrt{5}$ and $+\sqrt{5}$ at y_n . These differ. Since $y_n \rightarrow 0^+$, every punctured target interval contains such a cusp. Removing isolated points away from zero cannot repair the claimed germ; there are infinitely many of them accumulating at the endpoint. $\square$

An almost-everywhere derivative of A can still be bounded. That is not the classical derivative capability used by `D` and `SeriesDifferentiate`. The counterexample depends on this distinction, not on the size of the derivative where it happens to exist.

5.4 Prospective public trace

The exact-source characterization is:

```
s = AsymptoticAnalysis'AsymptoticFourierInverse[
  x + x^2 Sin[Log[x]], {x, 0}, {y, 2}];

d = AsymptoticAnalysis'SeriesDifferentiate[
  s, 1, "RemainderDerivativeOrder" -> 2];

a = AsymptoticAnalysis'SeriesObservable[d, Abs[z - 1], z];

AsymptoticAnalysis'SeriesDifferentiate[a]
```

The inspected source predicts the following metadata flow:

Object	Finite expression	Remainder information	Remaining derivative order
<code>s</code>	y	$\mathcal{O}(y^2)$	0 by default
<code>d</code>	1	$\mathcal{O}(y)$, after valid declaration	1
<code>a</code>	0	$\mathcal{O}(y)$, valid magnitude estimate	incorrectly retained 1
Next derivative	0	apparent $\mathcal{O}(1)$ derivative	should be refused

This is not a claim that the user declared nonexistent derivatives. The first declaration is justified by the exact source. The new nondifferentiability is introduced later by `Abs`. Nor is it an attempt to infer an arbitrary function from a malformed saved object: the source and every operation in the prospective test use public constructors.

The native test remains unexecuted. Its purpose is to verify the complete public route, including Fourier construction and representation recovery, while the source trace and Propositions 1–2 establish why inheriting that contract is mathematically unjustified.

6 Repairs: conservative first, richer only with proved contracts

6.1 Patch A: guard the signed-real shortcut

Immediately after the empty-row case in `fwdAbs`, the candidate patch requires all retained coefficient polynomials to be proved real under the recorded assumptions:

```
If[! AllTrue[T, realPolynomialQ[#[[2]], ell, ass] &],
  fail["UnprovedAbsoluteValueArgument",
    "The signed real Abs shortcut requires real retained coefficient polynomials.",
    <|"Reason" -> "RetainedCoefficientsNotProvedReal"|>]];
```

This preserves the ordinary eventual-sign shortcut for real jets. It rejects the complex perturbations in ABS-01 before an outer sum can erase the evidence. The error name intentionally says “unproved”: failure of a symbolic proof is not itself a proof of nonrealness.

The check is not a complete complex-modulus implementation. For example, $|1 + iu|$ is perfectly well defined and has an ordinary real Taylor expansion. The initial repair should return a truthful structured refusal rather than keep the current false expression. The final-result real-coefficient validator remains useful and should remain in place.

6.2 Patch B: lower the derivative contract at the observable boundary

A precision-preserving magnitude transformation need not be a regularity-preserving transformation. The distributed patch therefore lowers the derivative contract for a nonexact observable containing a variable-dependent `Abs`:

```
If[j[[2]] != Infinity &&
  ! FreeQ[body, node_Abs /; ! FreeQ[node, x]],
  d = Join[d, <|"RemainderDerivativeOrder" -> 0|>]];
```

This is deliberately conservative. It also lowers the contract in some safe cases, such as a modulus argument already bounded away from zero. Its advantage is a small, auditable change that blocks the reported inheritance path without changing the valid magnitude estimate or inventing a differentiability proof.

A subsequent refinement should preserve derivatives only after proving the relevant modulus arguments stay away from zero, or after proving a special exact structure at their zeros. Merely observing a zero retained expression, or merely proving that the complete argument is real, does not suffice. The prototype does not claim to have audited and redesigned every other caller of the internal jet evaluator.

Exact outputs are excluded from this conservative downgrade by the finite-tail condition. With the realness guard in place, an exact finite real jet accepted by the sign shortcut has an eventual sign (or is exactly zero); its elementary modulus identity is then valid on the relevant tail.

6.3 A richer value algorithm: square the norm, then take the positive root

Suppose

$$f(u) = u^\alpha(c + \varepsilon(u)), \quad c \in \mathbb{C} \setminus \{0\}, \quad \varepsilon(u) \rightarrow 0,$$

and the known jet has a remainder of magnitude $\mathcal{O}(u^P)$ with $P > \alpha$. For positive u ,

$$|f(u)| = \sqrt{f(u)\overline{f(u)}}. \tag{9}$$

The squared modulus has positive leading coefficient $|c|^2$, so a real positive-root expansion can be made in an ordinary unit series. Complex intermediate coefficients are handled by conjugation, not by pretending they are real.

The precision calculation is important. If $f = A + r$, $A = \mathcal{O}(u^\alpha)$, and $r = \mathcal{O}(u^P)$, then

$$f\bar{f} - A\bar{A} = A\bar{r} + \bar{A}r + r\bar{r} = \mathcal{O}(u^{\alpha+P}),$$

because $P > \alpha$. Taking the positive square root divides the local error scale by u^α , returning a modulus error of order $\mathcal{O}(u^P)$. Alternatively, the same magnitude precision follows directly from (1). Neither proof licenses derivative inheritance through an uncertain zero.

For a log-free normalized squared-modulus series

$$Q(u) = \sum_{k \geq 0} q_k u^k, \quad q_0 > 0,$$

write $B(u)^2 = Q(u)$ with $B(u) = \sum b_k u^k$ and $b_0 = \sqrt{q_0}$. Then

$$b_k = \frac{q_k - \sum_{j=1}^{k-1} b_j b_{k-j}}{2b_0}, \quad k \geq 1. \quad (10)$$

This is the recurrence implemented in the supplied Python reference. It correctly produces

$$|1 + iu| = 1 + \frac{1}{2}u^2 - \frac{1}{8}u^4 + \mathcal{O}(u^6).$$

The log-leading witness (4) requires additional coefficient algebra; the log-free reference deliberately refuses that input rather than disguise the missing reciprocal-log expansion.

6.4 Recovering regularity without overclaiming it

There are two useful admissible cases. If a real argument has a proved nonzero eventual sign and all requisite derivative bounds are known, **Abs** reduces to multiplication by that sign. If a complex argument stays away from zero relative to a nonzero leading term, the squared-modulus construction is smooth as a function of its real and imaginary components; derivative transport can be justified by the chain rule with those component bounds.

An argument known only as a tail approaching zero is a different case. Its zeros may be absent, finite, repeated, or infinite and simple. Magnitude data alone does not distinguish these possibilities. In that case the implementation should retain the value bound, lower the derivative capability, and explain why a subsequent classical derivative request is refused. This behavior addresses a concrete user-experience problem: the user should not be asked to trust derivative metadata that an intervening operation has silently invalidated.

7 Artifacts, test results, and implementation costs

7.1 What is supplied

The archive contains the article source and PDF; `code/stage_patch.py`; `code/modulus_j et.py`; the independent Python test suite and its recorded output; nine prospective Wolfram regression tests; a native characterization script; machine-readable finding/provenance records; and reproduction instructions. No upstream repository copy, font files, compilation debris, or checksum files are included.

`stage_patch.py` requires a checkout at the pinned revision and no local changes to the two edited files. It validates all exact edit anchors before staging changed modular files in a new

external directory. It does not overwrite the input checkout, does not produce a complete runnable package, and does not regenerate the standalone automatically. Its tests exercised exact-anchor fixtures, including missing anchors, duplicate anchors, and attempted reapplication; they were not tests of a complete modified repository.

7.2 Independent checks actually run

The supplied test runner completed **18 test methods with zero failures and zero errors**, using Python 3.13.5 and SymPy 1.14.0. The receipt records the precise local versions, counts, and scope. The tests include 27 conjugate-pair coefficient cases, 27 norm-square reference cases, 12 exact Fourier cusp phases, seven fixed derivative orders for a log-periodic control, and 30 zero-bracket controls for an additional smooth oscillatory family.

These counts are not 18 native package tests. In particular, none of the nine Wolfram tests was executed, and no Wolfram `TestReport` has been fabricated. Several attempts to obtain a native kernel returned HTTP 502/upstream service errors. The returned failure, rather than an inferred kernel result, is the execution evidence.

The reference checks verify the square-root recurrence by recomputing the squared output and comparing it with the exact squared modulus below the advertised cutoff. They also test negative and imaginary leading coefficients, integer Laurent shifts, an inherited error-power cap, rejection of inexact or unsupported input, and resource-budget failures. This provides useful independent evidence for the reference algorithm, not a certification of the Wolfram patch.

7.3 Scope and complexity of the reference implementation

`modulus_jet.py` accepts integer exponent keys and exact Gaussian-rational coefficients. It supports log-free polynomial and Laurent jets in a positive local variable, with an optional integer input-remainder power. It deliberately assigns no derivative capability. A finite input polynomial is not treated as proof that its modulus is a finite polynomial.

Let m be the number of relevant retained coefficients and N the relative Taylor depth. The implementation uses a cutoff-pruned norm-square convolution, followed by (10). Its elementary arithmetic-operation count is at most of order $m^2 + N^2$; exact coefficient bit complexity and simplification costs are additional. Explicit order and retained-pair limits bound the model's work. No wall-clock comparison against the Wolfram package was performed.

The conservative Wolfram guard is cheaper than building a modulus expansion but requires realness checks over the retained coefficient polynomials. This additional work is the price of preventing an invalid shortcut. A future implementation should reuse already established per-argument realness evidence where available, but should never reuse a final-output realness result as evidence about a distinct intermediate modulus argument.

The richer reference routine is research code with a deliberately bounded domain. It does not implement symbolic Gaussian coefficients under arbitrary assumptions, fractional exponents, polynomial-log leading blocks, reciprocal-log coefficients, Mathics integration, or proof-producing derivative transport. These are explicit limitations of the supplied artifact, not newly asserted shortcomings of the upstream project.

8 Focused implementation and acceptance plan

8.1 First milestone: remove false successes

Apply the two conservative source edits to a separate pinned checkout, regenerate the standalone with the repository’s existing builder, and run the supplied focused tests in a fresh native kernel. The acceptance conditions are specific: the complex-unit and complex-log witnesses must not return the source-predicted false exact answers; the uncertain-modulus derivative request must not succeed on an inherited classical derivative contract; and ordinary real-sign controls must retain their correct values.

The current policy tests expect the candidate’s structured refusal for complex retained coefficients. A richer correct modulus implementation may replace that refusal with the correct expansion and corresponding tail. Such a change should update those policy assertions rather than being treated as a regression merely because it is more capable.

8.2 Second milestone: restore justified capabilities

Implement the norm-square construction for nonzero-leading log-free jets, with the precision rule proved above. Start with exact rational or Gaussian-rational coefficients and integer weights; this is a small domain for which independent exact oracles are supplied. Preserve the distinction between exactness of the input finite expression and exact termination of its modulus expansion.

At the same time, refine the conservative derivative downgrade only where there is a proof that each relevant modulus argument remains away from zero, or an exact local factorization proves sufficient smoothness at its zeros. The exact Fourier witness should remain a negative control. Testing only positive units would miss the boundary that caused ABS-02.

8.3 Third milestone: make the new behavior understandable

Document the distinction between a real final result and a real intermediate argument of **Abs**. A useful example pair is

$$|(1 + iu) + (1 - iu)| = 2, \quad |1 + iu| + |1 - iu| = 2\sqrt{1 + u^2}.$$

The first permits cancellation before applying the nonlinear operation; the second does not.

For derivative refusal, the diagnostic should say that the magnitude remainder survived the modulus but classical differentiability did not follow. The user can then provide a separately justified regularity theorem or choose a supported nonzero-sign branch. It should not suggest that requesting more terms alone necessarily repairs infinitely many accumulating zeros.

These milestones are confined to the newly demonstrated contracts. The existing release, compatibility, scaling, and architectural backlog remains in the upstream registers and is not duplicated here.

9 Limitations and conclusion

This audit supplies two new source-grounded counterexamples, not a claim that every repository path has been verified. It does not report observed native package outputs, a native patch pass, a full-suite pass, a Mathics compatibility result, a measured performance regression, or a speedup. The native characterization scripts and tests are provided precisely because those integration observations remain to be collected.

The first finding is particularly easy to miss when testing only final realness: the invalid complex intermediate pieces cancel, and the wrong result becomes an apparently harmless real constant or logarithm. The second shows why a correct magnitude estimate is not enough to preserve every capability attached to an asymptotic object. A smooth exact Fourier source provides valid input derivative bounds, yet a later modulus introduces cusps at infinitely many points approaching the endpoint.

The immediate recommendation is therefore narrow: stop the two false-success paths, retain valid magnitude information, and regain richer modulus/derivative behavior only through explicit mathematical transfer rules. The supplied conservative patch, exact reference implementation, and focused tests make those obligations concrete without reissuing the previous review backlog.

A Reproduction and interpretation

A.1 Independent checks

From the archive root, with Python and SymPy installed:

```
python tests/test_independent.py
```

This rewrites `evidence/independent_checks.json` with the new local run. The distributed text log records the authoring run. No upstream package is loaded.

A small reference-algorithm example is:

```
import sys
sys.path.insert(0, "code")
import sympy as sp
from modulus_jet import modulus_jet
u = sp.Symbol("u", positive=True)
result = modulus_jet({0: 1, 1: sp.I}, 6)
print(result.expression(u)) # 1 + u**2/2 - u**4/8
print(result.error_power)   # 6; magnitude remainder O(u**6)
```

A.2 Native characterization, not a recorded result

```
wolframscript -file code/native_characterizations.wl \
/path/to/pinned/AsymptoticAnalysis.wl
```

The script prints its kernel version, finite expressions, remainders, derivative-contract fields, and failures. Run it against the unmodified snapshot and a separately staged candidate. Do not reinterpret these unexecuted script contents as observations.

For the prospective test file, load the package first in a fresh kernel:

```
Get["/path/to/AsymptoticAnalysis.wl"];
TestReport["/path/to/tests/AbsObservableDelta.wlt"]
```

The test file includes candidate-policy assertions as well as correctness controls. It is not the repository's complete suite.

A.3 Stage the source edits

```
python code/stage_patch.py \
/path/to/pinned-checkout /path/to/new-external-staging
```

The output contains only the changed `src/Kernel/AsymptoticAnalysis.wl` and `src/Kernel/SeriesOperations.wl`. Review the changes and transfer them to a separate complete checkout. Regenerate the root standalone using the repository’s existing `validation/build_standalone.py`. No standalone artifact produced by this review has been executed or certified.

B Source locator and evidence register

Obligation	Pinned source location
Real coefficient boundary	<code>src/Kernel/AsymptoticAnalysis.wl</code> , lines 244–267
Shared modulus shortcut	<code>src/Kernel/AsymptoticAnalysis.wl</code> , lines 437–465
Observable argument admission and dispatch	<code>src/Kernel/SeriesOperations.wl</code> , lines 272–346
Observable representation inheritance	<code>src/Kernel/SeriesOperations.wl</code> , lines 348–369
Classical derivative gate and propagation	<code>src/Kernel/SeriesOperations.wl</code> , lines 535–558
Recovery of the finite Fourier approximation	<code>src/Kernel/SeriesOperations.wl</code> , lines 40–67
Fourier constructor’s result and derivative field	<code>src/Kernel/FourierCoefficients.wl</code> , lines 205–237

The accompanying `evidence/findings.json` separates source deductions, independent checks, unexecuted native probes, and patch scope. `evidence/provenance.json` records the snapshot and source-review limitations. Commit identities serve as provenance; no checksum files are included.

References

- [1] VladimirReshetnikov/Asymptotic. Reviewed commit:
651f2029d0b2cd4da9e4dfdf1f4275a124d23b99.
<https://github.com/VladimirReshetnikov/Asymptotic/commit/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99>
- [2] AsymptoticAnalysis core, pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/AsymptoticAnalysis.wl>

- [3] Series operations, pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/SeriesOperations.wl>
- [4] Fourier coefficient/inverse implementation, pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/FourierCoefficients.wl>
- [5] Prior-review index, pinned revision.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/README.md>
- [6] Consolidated code-review status, pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/CODE_REVIEW_STATUS.md
- [7] Wave-3 intake, pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_3_INTAKE.md
- [8] Wave-4 intake, pinned revision.
https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_4_INTAKE.md
- [9] Review 1, original finding ledger.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-1/evidence/findings.csv>
- [10] Review 4, original finding ledger, including R01 and R02.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-4/evidence/findings.json>
- [11] Review 5, original finding ledger.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-5/evidence/findings.json>
- [12] Review 6, original finding ledger.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-6/evidence/findings.csv>
- [13] Review 7, original finding ledger.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-7/evidence/findings.csv>
- [14] Review 8, original finding ledger.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-1/code-review-8/evidence/findings.csv>
- [15] Review 25, original summary and evidence limits.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-3/code-review-25/README.md>